

System Design Basics

Lesson 2: Understanding Requirements

Understanding Requirements - Study Notes

Key Concepts

1. **Functional Requirements** – What the system must do (e.g., process payments, generate reports).
2. **Non Functional Requirements** – How the system performs (e.g., uptime, latency, scalability).
3. **Requirements Traceability** – Linking requirements to design, implementation, and testing artifacts.
4. **Stakeholder Prioritization** – Ranking requirements based on business value, risk, and feasibility.
5. **Acceptance Criteria** – Clear, testable conditions that determine when a requirement is satisfied.

Summary

Requirements engineering is the foundation of any successful software project. Functional requirements describe the specific behaviors and features the system must provide, while non functional requirements (NFRs) set the quality attributes such as performance, reliability, and security. A well defined set of requirements ensures that all stakeholders—product owners, developers, testers, and customers—share a common understanding of what the product should achieve.

Effective requirement capture involves eliciting input from diverse stakeholders, documenting the needs in clear, concise language, and maintaining traceability throughout the development lifecycle.

Prioritization and validation are critical: high impact, low risk requirements should be addressed first, and each requirement must have measurable acceptance criteria to guarantee that the final product meets expectations.

Important Points

- **Separate Functional & Non Functional** – Keep them in distinct sections to avoid confusion.
- **Use SMART Criteria** – Requirements should be Specific, Measurable, Achievable, Relevant, and Time bound.
- **Quantify NFRs** – Example: “System uptime "e /99.99 /% over a calendar month” or “Maximum response time "d /200 /ms for 95 /% of requests.”
- **Document Acceptance Criteria** – Define testable conditions (e.g., “Given X, when Y, then Z”).
- **Maintain Traceability** – Link each requirement to design elements, code modules, and test cases.
- **Iterate & Review** – Regularly revisit requirements with stakeholders to capture changes and validate assumptions.
- **Use Templates & Tools** – Requirement management tools (Jira, Azure Boards, Confluence) help keep consistency and visibility.

Examples

1. ****Functional Requirement****

“The system shall allow users to reset their password via email.”

Acceptance Criteria:

- User clicks “Forgot Password” !’ receives a unique, time limited reset link.
- Reset link expires after 30 minutes.
- Password must be at least 12 characters, include a number and a special character.

2. ****Non Functional Requirement****

“The API shall support a throughput of 10,000 queries per second (QPS) during peak hours.”

Acceptance Criteria:

- Load testing shows sustained 10,000 QPS for 1 hour with < 1 % error rate.
- Latency stays below 150 ms for 95 % of requests.
- System recovers automatically within 5 minutes after a simulated outage.

Quick Review

1. What is the difference between functional and non functional requirements?
2. Why is traceability important in requirements engineering?
3. Give an example of a measurable non functional requirement for a web service.
4. What does the “SMART” acronym stand for, and why is it useful?
5. How can acceptance criteria help prevent scope creep?

Further Reading

- ****Requirements Engineering Body of Knowledge (REBOK)**** – Comprehensive framework for RE practices.
- ****IEEE 830-1998**** – Standard for Software Requirements Specifications.
- ****Agile User Stories & Acceptance Criteria**** – Techniques for lightweight requirement capture.
- ****Performance Testing Best Practices**** – Tools and methods for validating QPS, latency, and uptime.
- ****Risk Based Requirements Prioritization**** – Strategies to balance business value and technical risk.

